



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS 216 Practical Assignment 4

- Date Issued: **7 April 2026**
 - Date Due: **11 May 2026** before **08:00**
 - Submission Procedure: **Upload to the web server (wheatley) + clickUP**
 - This assignment consists of **6 tasks** for a total of **125 marks**.
-

1 Introduction

During this practical, you will be completing the functionality of your flight booking website and accompanying API. After completing this assignment, your website will make requests to the API you created in the previous practical. A user will be able to view planes, add them to their favourite planes but now, the API will return persistent data stored in your database. Users will also be able to book flights.

2 Constraints

1. You must complete this assignment individually.
2. You may ask the Teaching Assistants for help but they will not be able to give you the solutions.
3. You must produce all of the source files yourself; you may not use any tool to generate source files or fragments thereof automatically (**This includes ChatGPT, Gemini etc.!!**).
4. Your assignment will be viewed using Google Chrome so be sure to test your assignment in this browser. Nevertheless, you should take care to follow published standards and make sure that your assignment works in as many browsers as possible.
5. You may utilise any text editor or IDE, upon an OS of your choice, again, as long as you do not make use of any tools to generate your assignment. (**This includes ChatGPT, Gemini etc.!!**)
6. All written code should contain comments including your name, surname and student number at the top of each file.
7. Your assignment must work on the **wheatley** web server, as you will be marked off there.
8. **You may not use external libraries to perform security operations (You may use PHP built-in functionality).**
9. **Server-side scripting should be done using an Object Oriented approach.**

3 Submission Instructions

You are required to upload all your source files (e.g. HTML5 documents, any images, etc.) to the web server (**wheatley**) and clickUP in a compressed (zip) archive. Make sure that you test your submission to the web server thoroughly. All the menu items, links, buttons, *etc.* must work and all your images must load. Make sure that your practical assignment works on the web server before the deadline. No late submissions will be accepted, so make sure you upload in good time. The server will not be accepting any uploads and updates to files from the stipulated deadline time until the end of the marking week (Thursday at 3pm).

Note, wheatley is currently available from anywhere. But do not rely that outside access from the UP network will always work as intended. You must therefore make sure that you **ftp** your assignment to the web server. Also make sure that you do this in good time. A snapshot of the web server will be taken just after the submission was due and only files in the snapshot will be marked.

Practicals are marked by demonstrating your practical to a tutor during the allotted marking weeks. If you do not demonstrate your practical you will receive 0 for the practical. You will only be marked in the practical session that you have booked on the cs portal, if you miss it, you will receive 0 (Unless special permissions have been granted by the lecturer. i.e. You were sick and able to provide a sick note).

If your assignment is not uploaded to ClickUp that will also result in you receiving 0.

NB: You must also submit a ReadMe.txt file.

It should detail the following:

- how to use your website
- default login details (username and password) for a user you have on your API
- any functionality not implemented
- bonus features that you have implemented

You need to add a link to this file from your login page.

4 Online resources

PHP Sessions - http://www.w3schools.com/php/php_sessions.asp

PHPMyAdmin - http://www.phpmyadmin.net/home_page/index.php

Timestamps - https://en.wikipedia.org/wiki/Unix_time

Cookie - https://www.w3schools.com/js/js_cookies.asp, <https://developer.mozilla.org/en-US/docs/Web/API/Document/cookie>

Local Storage - https://www.w3schools.com/jsref/prop_win_localstorage.asp

5 Rubric for marking

PHP API	25
Book flights page	10
Planes Page	10
View Page	5
Login & Logout	15
HTML	5
Security	5
API + Validation	5
Local DOM Storage	10
API Key	3
JS	4
Explanation	3
Favourites API	25
API + SQL	10
Authorization + Validation	10
Favourites Page	5
Book Flights API	50
API	25
SQL	10
Database	10
Authorization + Validation	5
Upload	
Does not work on wheatley	-20
Not uploaded to clickUP	-125
Not demoed	-125
Bonus	10
Total	125

6 Assignment Instructions

Task 1: Integrating your PHP API (25 marks)

For this task, you should alter your **Book Flights**, **Bookings**, **Planes**, **Favourites** and **view** pages so that they makes use of your own API that you created in PA3 and the rest you will implement in this PA instead of the Wheatley API used in PA2. The filter/search/sort should now work using your PHP API.

Task 2: Login & Logout (15 marks)

This task requires you to implement login and logout functionality for your website. Implement the [login API type](#) for your PHP API. You should return the API key of the user once they logged in successfully. Your API should follow the following structure:

Request Example You need to follow this structure/parameteres when making a login request.

Example Post body:

```
{
    "type": "Login",
    "email": "tony@starkindustries.com",
    "password": "LoveYou3000"
}
```

Response Object

Your API should return a structured JSON Object as a response. Here is what the response to the request example given above should be:

```
{
    "status": "success",
    "timestamp": "1679507636541",
    "data": [
        {
            "apikey": "5c331d9c15d564d3d0de0f1f2937b92e"
        }
    ]
}
```

You can also include other information such as user name, email, etc After implementing the login functionality in the API, make a basic login page in `login.php`.

NB: Once a user has successfully logged in, their [API key needs to be returned](#), and any API requests need to use this API key (for retrieving product information). You will need to store this in a cookie or local DOM storage as seen in Task 3. Ensure that your API only accepts valid requests.

The second part of this task requires you to implement [logout](#) functionality which simply clears and removes the cookie/session. (Add this functionality to `logout.php`)

Remember, that the logout option should only be displayed to users that are logged in.

Task 3: Cookie or Local DOM Storage (10 marks)

Storing the API key in the cookie or local DOM storage makes it easier to retrieve the key to make the requests to your PHP API. Once a user has logged in, you should [use either a cookie or local DOM storage to store the API key](#) that was returned during the login process. **You need to be able to explain your choice between using a cookie or local storage.**

Task 4: favourites API (20 marks)

For this task, you will need to implement the "favourite" type for the API. This task consist of two parts:

- Adding to favourites:
When a plane is added to a user's favourites, it should update your database and be displayed on the "Favourites" page.
- Removing from favourites:
When a plane is removed from a user's favourites, it should update your database and should not be displayed on the "Favourites" page anymore.

You will also need to modify your database design. You can create a new table with favourited planes for each user. You can decide on the schema yourself and explain it in the read me

Additionally, please note the following:

- You should add a "favourites" API type to your PHP API. It is up to you if you want to handle the "remove from favourites" through the same type or create a new one.
- **Only registered users can "add to favourites"**. Therefore the API key must be a required parameter.
- Once a user logs in and navigates to Favourites, they should see only the planes they have added to their favourites (keep this in mind when making your DB table).

Task 5: Book Flights API (50 marks)

NB! Update the arrival date on the 'Book Flight' page to indicate the return date!

For this task, implement the "BookFlight" and "GetBookings" types for the API.

Since flights only store a departure_date (as described in the schema below), booking a return trip requires creating two separate flights and two bookings.

How you structure your requests and your responses are not important however you should structure them in a way that seems logical

Book flight

BookFlights type has two parts:

- Booking a flight:
Clicking the button on the Book Flight page should save the flight and display it on the bookings page.
- Removing a flight:
Removing a flight from a user's bookings should update the database and remove it from the "bookings" page.

You will need to modify your database design to add two new tables called bookings and flights.

Bookings will store each flight for the user

Flights is used to store individual flights

Below are the schema for the tables: **Flights Table**

- id (PK)
- plane_id (FK)
- departure_airport_code (FK)
- arrival_airport_code (FK)
- departure_date
- flight_time
- distance

Take note of the following calculations

- To calculate the distance between two airports, use the Haversine formula: Haversine formula.

$$\text{hav}(\theta) = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)$$

$$\theta = 2 \arcsin\left(\sqrt{\text{hav}(\theta)}\right)$$

$$d = R \cdot \theta$$

Where

$$\begin{aligned} \varphi_1, \varphi_2 &= \text{latitudes in radians} \\ \lambda_1, \lambda_2 &= \text{longitudes in radians} \\ \Delta\varphi &= \varphi_2 - \varphi_1 \\ \Delta\lambda &= \lambda_2 - \lambda_1 \\ R &= \text{Earth radius } (\approx 6377 \text{ km}) \\ d &= \text{distance between points} \end{aligned}$$

- To calculate the flight time, use the following formula

$$v_c = v_{\max} \cdot \left(1 - 0.2 \frac{C_{\max}}{C_{\max} + 80 \cdot \text{seats}}\right)$$

$$t_{\text{climb/descent}} = t_{\text{climb_base}} \cdot \left(1 - e^{-k \cdot d}\right)$$

$$t_{\text{climb_base}} = \begin{cases} 20 \text{ min,} & \text{if seats} > 300 \\ 15 \text{ min,} & \text{if } 200 < \text{seats} \leq 300 \\ 10 \text{ min,} & \text{if } 100 < \text{seats} \leq 200 \\ 7 \text{ min,} & \text{if } 50 < \text{seats} \leq 100 \\ 5 \text{ min,} & \text{if seats} \leq 50 \end{cases}$$

$$t_{\text{total}} = \frac{d}{v_c} + t_{\text{climb/descent}} + 15_{\text{min}}$$

Where

$$\begin{aligned} d &= \text{Distance_km} \\ v_{\max} &= \text{Planes max_speed_kmh} \\ R_{\max} &= \text{Planes max_range_km} \\ C_{\max} &= \text{Planes max_cargo_kg} \\ \text{seats} &= \text{Planes number of seats} \end{aligned}$$

Bookings Table

- id (PK)
- flight_id (FK)
- user_id (FK)
- passengers

This table will link users to which flight they have booked for.

Additionally, please note the following:

- When booking a flight:
 - First, check if a matching flight already exists in the database (match by departure airport, plane ID, and departure date) and display it on the Book Flights page.
 - If the user is booking a return trip, also check for an existing return flight.
 - If matching flights are found, allow the user to select one or both flights for booking. Otherwise if a flight doesn't exist create one in the database.
 - Retrieve all existing bookings for the selected flight(s) and calculate the total number of passengers.
 - Compare the total passengers to the plane's seat capacity:
 - * If seats are available, proceed with booking.
 - * If the flight is full, notify the user and suggest choosing a different departure date/return date or plane.
 - Confirm the booking and update the database with the new booking(s).
 - Update the bookings page immediately to reflect the user's selection.

GetBookings

This task requires you to retrieve all booked flights for the relevant user.

Task 6: Bonus (10 marks)

You may add additional 'nice to have' features and depending on the level of difficulty marks will be given.

Note: Any Bonus Marks received will be capped to full marks.